



Tutorial 01

EE-809 Analog IC Design

Introduction to Cadence

Release: 1.0

Date: 07-Sept-2026



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.



Contents

Objectives	5
Introduction	5
Analog IC Design Flow and Required Tools	5
Logging-In/Logging-Out.....	7
Using the Linux operating system.....	7
Task manager in Linux.....	7
Cadence	7
Starting Cadence	8
Creating a New Library	9
Creating a Cell.....	10
Working with Schematic Editor	10
Simulating a schematic	14
Introduction to TSMC65N PDK	15
RF_NMOS six terminal	16
RF_PMOS five terminal	16
RF_NMOS four terminal	16
RF_PMOS four terminal	17
NMOS four terminal	17
PMOS four terminal	17
NMOS three terminal.....	17
PMOS three terminal	17
NPN BJT three terminal.....	18
PNP BJT three terminal	18
ESD_Diode	18
ESD_Diode three terminal.....	18
Diode 2 terminal.....	18
Resistor 2 terminal.....	19
Resistor three terminal.....	19
Varactor two terminal	19
Varactor three terminal.....	19
Inverter gate.....	19
NAND-2T gate.....	20
NAND-3T gate.....	20
NAND-4T gate.....	20
NOR-2T gate	20



NOR-3T gate	20
NOR-4T gate	21
Pass gate	21
Tri-state inverter gate	21
Devices available in TSMC65nm PDK.....	21
Summary.....	23
Exercise	24



Objectives

In this exercise, the aim of this lab will be

- To become familiar with custom IC Design flow
- To login, start a shell tool and start the Cadence Virtuoso software
- To learn about the PDK and add the PDK library to the library manager
- To create a working library and get familiar with technology
- To know about the short/bind keys that are used in the complete flow

Introduction

This manual is intended to introduce **Analog and Mixed Signal IC Designers** to the Cadence Design Environment, and to describe all the steps necessary for running the Cadence tools at the NUST Chip Design Centre.

Cadence is an Electronic Design Automation (EDA) environment that allows integrating different applications and tools in a single framework. This integration provides all the stages of IC design and verification from a single environment. These tools are completely general, supporting different fabrication technologies. When a particular technology is selected, a set of configurations and technology-related files are employed for customizing the Cadence environment. This set of files is commonly referred as a design kit. It is not the objective of this manual to provide an in-depth coverage of all the applications and tools available in Cadence. Instead, a detailed introduction to those required for an analog designer, from the conception of the circuit to its physical implementation, will be provided.

Analog IC Design Flow and Required Tools

Figure. 1 shows the basic design flow of an analog IC design, together with the Cadence tools required in each step.

First, a schematic view of the circuit is created using the Cadence Composer Schematic Editor.

Then, the circuit is simulated using the Cadence Affirma analog simulation environment. Different simulators can be employed, some sold with the Cadence software (e.g., Spectre) some from other vendors (e.g., HSPICE) if they are installed and licensed.

Once circuit specifications are fulfilled in simulation, the circuit layout is created using the Virtuoso Layout Editor.

The resulting layout must verify some geometric rules dependent on the technology (design rules). For enforcing it, a Design Rule Check (DRC) is performed. Optionally, some electrical errors (e.g. shorts) can also be detected using an Electrical Rule Check (ERC). Then, the layout should be compared to the circuit schematic to ensure that the intended functionality is implemented. This can be done with a Layout Versus Schematic (LVS) check. All these verification tools are included in the PVS, Pegasus software in Cadence (more powerful Cadence tools can also be available, like Dracula, or Assura in deep submicron technologies).



Finally, a netlist including all layout parasitics should be extracted using Quantus or Calibre, and a final simulation of this netlist should be made. This is called a post-layout simulation, and is performed with the same Cadence simulation tools.

Once verified the layout functionality, the final layout is converted to a certain standard file format depending on the foundry (GDSII, CIF, etc.) using the Cadence conversion tools.

To effectively design a circuit, the behavior of the circuit elements must be known. This lab will give a walk-through in characterizing NMOS transistor using various Cadence tools. To make things concrete in the end, it will also be required to complete tasks in a similar way as will be shown in the following exercise steps.

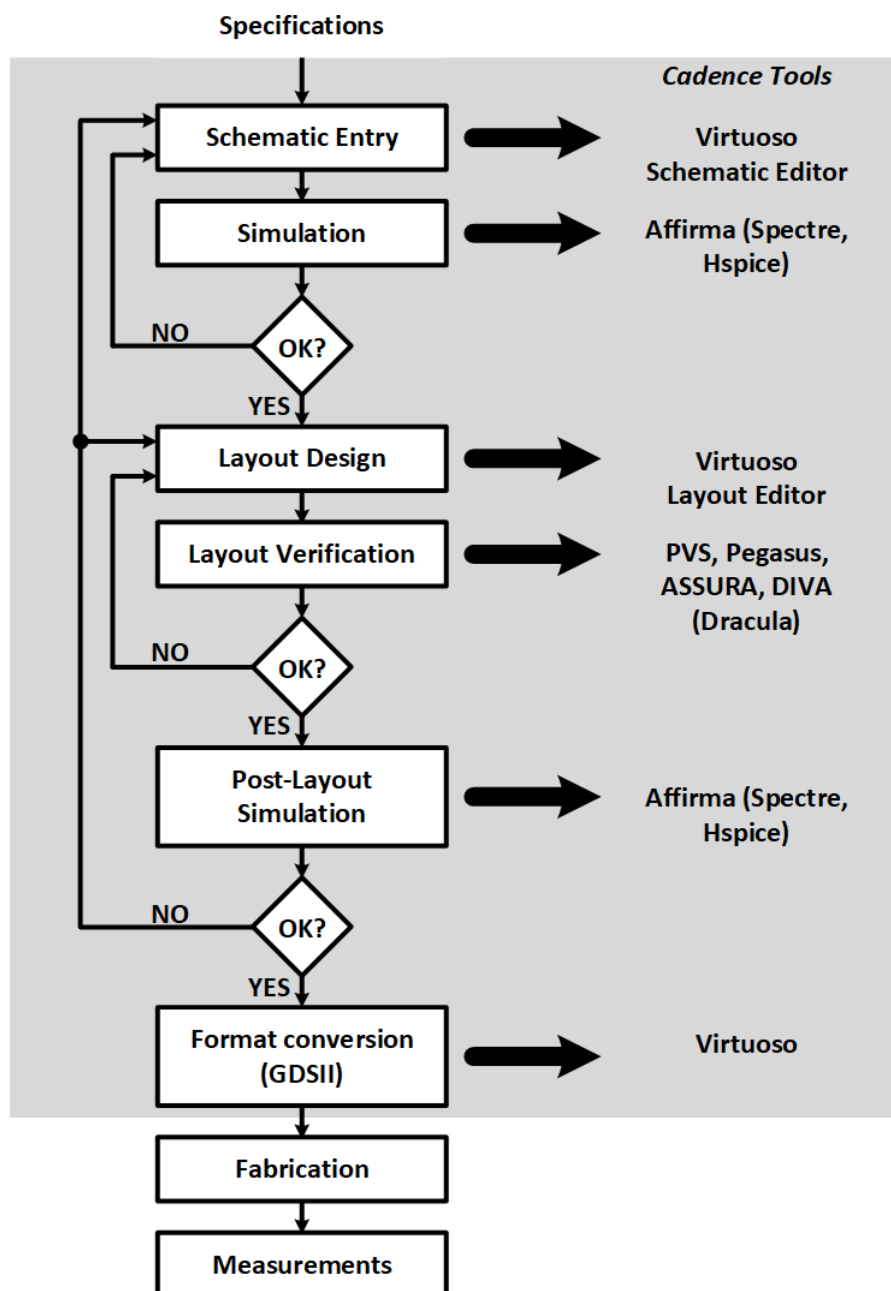


Figure 1: Custom IC Design Flow and Cadence tools involved



Logging-In/Logging-Out

In order to use the Linux machines, you must first login to the system. As with the PC lab, login using login credentials provided by NCDC. This will be your NCDC Linux account and all files will be stored in the generated work directory even after logging-out.

Using the Linux operating system

Using the Linux operating system is like using other operating systems such as DOS. Linux commands are issued to the system by typing them into a "shell" or "xterm". Linux commands are case sensitive so be careful when issuing a command, usually they are given in lower-case.

The following list (Table 1-1) summarizes some basic commands required to manage the data files you will be creating in this lab course. All Linux commands are entered from the shell or xterm window. Do not use Linux commands for modifying, deleting, or moving any Cadence data files.

Commands	Descriptions
ls -al	Lists all files in current directory, including hidden files
ll	Lists all files in current directory with time stamps and permissions
mkdir XXXXX	Creates a new directory titled "XXXX"
cd XXXX	Changes the current directory to "XXXX"
cd..	Changes the current directory back one level
cd~	Changes the current directory back to the home directory
cp XXXX YYYY	Copies file "XXXX" to file "YYYY"
mv XXXX YYYY	Moves (or renames) file "XXXX" to file "YYYY"
rm XXXX	Deletes file "XXXX"
gedit XXXX &	Opens file "XXXX" using the text editor "gedit", "&" returns the prompt to the shell
clear	Clears the screen (alternatively, you can press Ctrl-L)
top	Shows the process/memory table (press q to exit)
\rm -rf XXXX	Removes the file or directory "XXXX" without any confirmation

Table 1: Common linux commands

Task manager in Linux

Task Manager in Linux If you type **"ps -ef | grep <NetID>"**, it will list the processes running on the system. If for some reason Cadence, Virtuoso or Spectre crashes or freezes, the process could still be running and slowing down the server without doing anything. Using the **"ps"** command above you can find the index number (in the first column of the list next to NetID) of the process. Now if you type **"kill -9 <index number>"**, you can kill the process. You can also issue the command **"kill -1 -1"** to kill all your processes.

Cadence

The Cadence Development System consists of a bundle of software packages such as schematic editors, simulators and layout editors. This software manages the development process for analog, digital and mixed-signal circuits. In this course, we will strictly use the tools associated with analog circuit design.

All the Cadence design tools are managed by a software package called the Design Framework II. This program supervises a common database which holds all circuit



information including schematics, layouts, and simulation data. From the Design Framework II, also known as the "framework", we can invoke a program called the "Library Manager" which governs the storage of circuit data. We can access libraries and the components of the libraries called cells.

Also, from the framework we can invoke the schematic entry editor called "Composer/schematic editor". Composer is used to draw circuit diagrams and draw circuit symbols. A program called "Virtuoso Layout Suite" is used for creating integrated circuit layouts. The layout is used to create the masks which are used in the integrated circuit fabrication process.

Finally, circuit simulation is handled through an interface called Analog Design Environment (ADE). This interface can be used to invoke various simulators including HSPICE, Spectre, UltraSim, and Verilog. We will be using the Spectre simulator in this course.

Starting Cadence

After logging into a Linux workstation in the lab using your ID and password, the first step is to open a **Terminal** using the menu item **Applications→Favorites→Terminal** (Figure 2) or using the right-click menu on the desktop. The next step is to create a new directory (Figure 3) that will keep all Cadence designs. This directory can be used only for one technology (for this course, TSMC_65nm). If you need to use another process for a different project or course, you need to create another directory.

Never start Cadence directly from your home directory, always change directory (cd) into the subdirectory that you generated corresponding to the technology you will use, then start Cadence as follows:

```
cd /home/ncdc_AMS_Cohort/your_work_directory/  
csh  
source cshrc  
virtuoso
```

This will load Cadence. The Command Interpreter Window (CIW) will now load as shown in Figure 4.

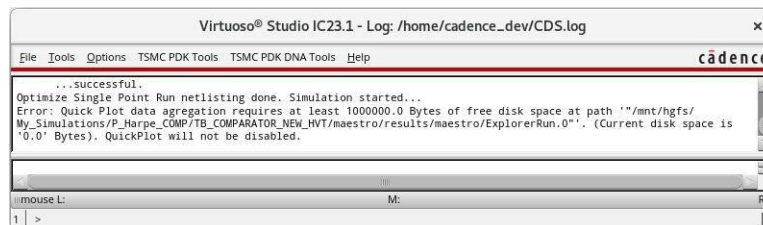


Figure 2: Terminal in Red-Hat



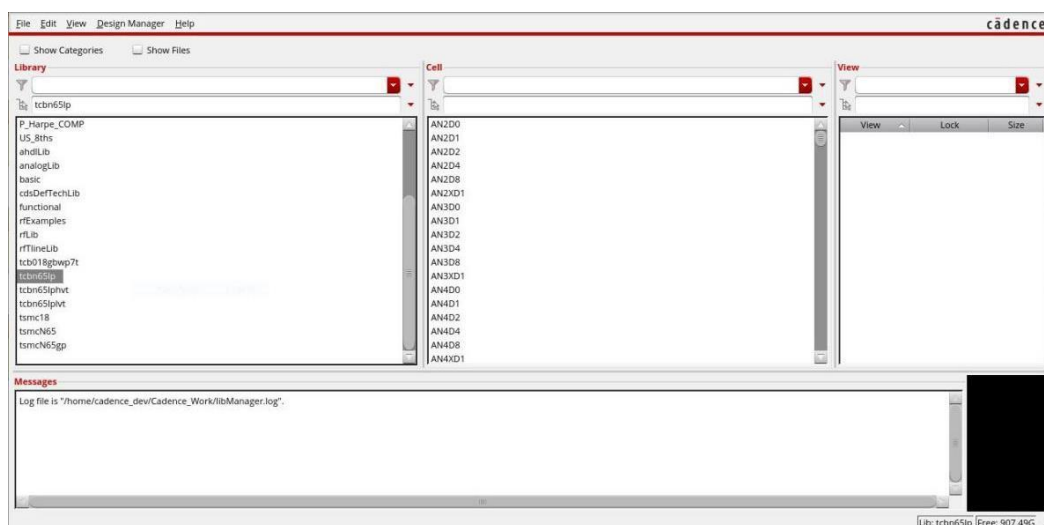
```
cadenceserver@ncdc-s-cd-01:~/AMS_Cohort
File Edit View Search Terminal Help
[cadenceserver@ncdc-s-cd-01 ~]# csh
[cadenceserver@ncdc-s-cd-01 ~]# source cshrc
*****
***** Welcome to Cadence Tools *****
*****
[cadenceserver@ncdc-s-cd-01 ~]# virtuoso
```

Figure 3: Terminal in Red-Hat



Creating a New Library

From the CIW (Figure 4) select **Tools**→**Library Manager** to load the library manager (Figure 5). The Library Manager stores all designs in a hierarchical manner. A library is a collection of cells. For example, if you had a digital circuits library named Digital, it will have several cells included in it. These cells will be inverters, nand gates, nor gates, multiplexers, etc. Each cell has different views. These views will be symbols, schematics, or layouts of each cell.





The first thing you need to do is to start a design and create a library to store the cells you will be designing in this lab. Let's call this library "Lab1". From the Library Manager window select **File→New→Library**. Name the library "**Lab1**" (without the quotes) and select **OK**. In the popup window that appears select "**Attach to an existing technology library**" (Figure 6) and select **OK**. In the next window make sure the **tsmc65N** Technology library is selected and select **OK**.

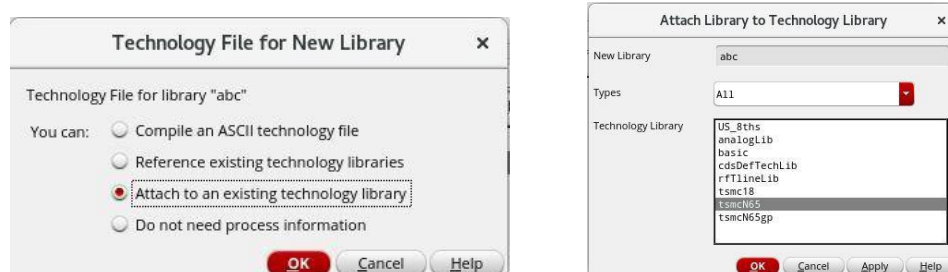


Figure 6: Attaching to an existing technology and selecting proper PDK

Creating a Cell

We will design a simple inverter as a first practice circuit. Select the library you want to put the cell into by clicking on it to highlight it, in this case "**Lab1**", and then **File→New→Cell View**. Name your cell **INV** or use **Camel Case** wording. The application you want to use here is Schematics L as seen in Figure 7. After selecting **OK** on the "New File" tab and **OK** on the next popup window, the schematic window opens.

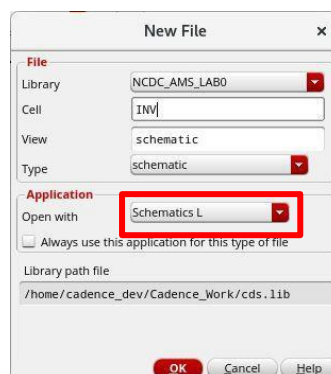


Figure 7: Creating a new schematic view

Working with Schematic Editor

We wish to add two transistors so that we can make an inverter. To do this we need to add an instance.

1. You can do this either by clicking **Create→Instance** or by pressing the "**I**" hotkey on the keyboard. A window titled "**Component Browser**" should pop up. Hit the "Browse" button, and make sure the library **tsmc65N** is selected.
2. Select **nch_mac** and make sure **symbol** is highlighted in the view section. A window with all the NFET parameters should pop up. The parameters may be edited if desired. Next, click "**hide**" and place the NFET on the schematic by just left click anywhere in the schematic editor window.
3. Go back to the Component Browser and select **pch_mac** and make sure the symbol is highlighted.



- Next, the components need to be connected together by wires. You can select **Add→Wire** or use the "**w**" hotkey.
- The parameters of the PFET need to be changed if they were not changed earlier. To change the parameters of a device use **Edit→Properties→Objects** or use the "**q**" hotkey after selecting the device. In order to compensate for the lower mobility in the PMOS transistor, the width of the PMOS transistor needs to be changed and for this lab make the width of PMOS 2X greater than NMOS (Hint: **PMOS W/L=1/0.1** & **NMOS W/L=0.5/0.1**) shown in figure 8.

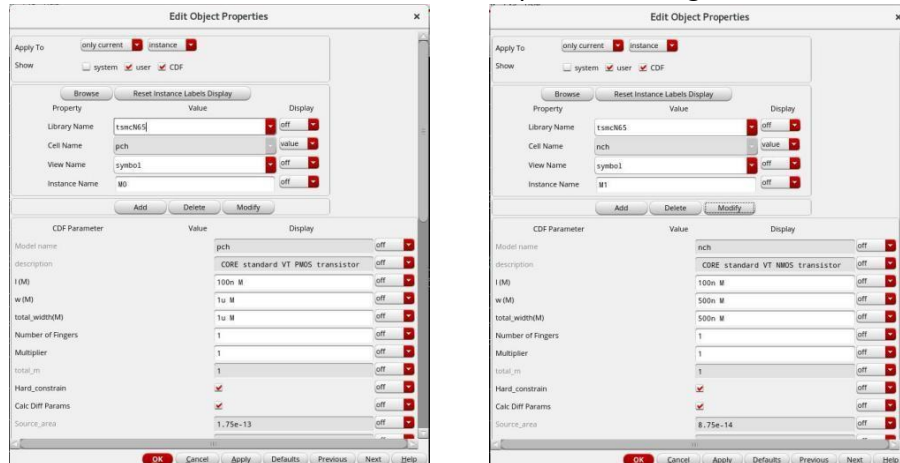


Figure 8: Width and length of PCH & NCH

- This can be done by methods previously mentioned or by selecting the element and editing its properties in the Property Editor Window in the lower left of the schematic window.
- Next, add sources to the circuit by adding an instance and browsing the "**analogLib**" library. The power supply and the input voltage sourced need to be added. For DC supply search "**vdc**" and type "**VDD**" in "**DC Voltage**" as shown in figure 9a. Similarly, for the input we want to give a step input, again search "**vpwl**" in **analogLib** and change its values as shown in figure 9b.

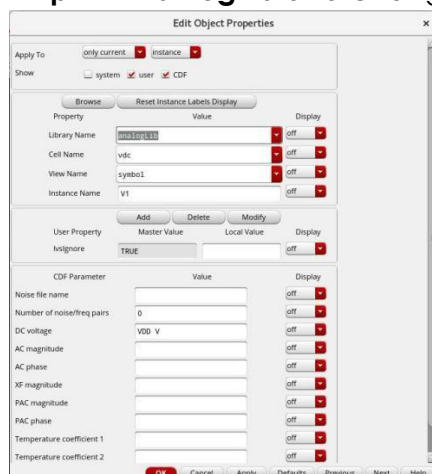


Figure 9a: vdc value setting

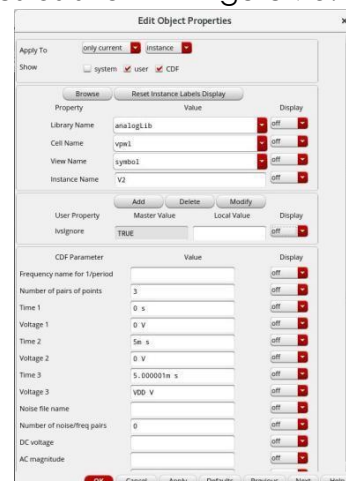


Figure 9b: vpwl value setting

- Label the wires by pressing short key "**I**", type **VIN** and click on the input wire also same for the output wire too. When all is complete it should resemble



Figure 10. Also, a summary of common schematic editor along with layout hotkeys and commands is shown in Table 2 which will be helpful in future.

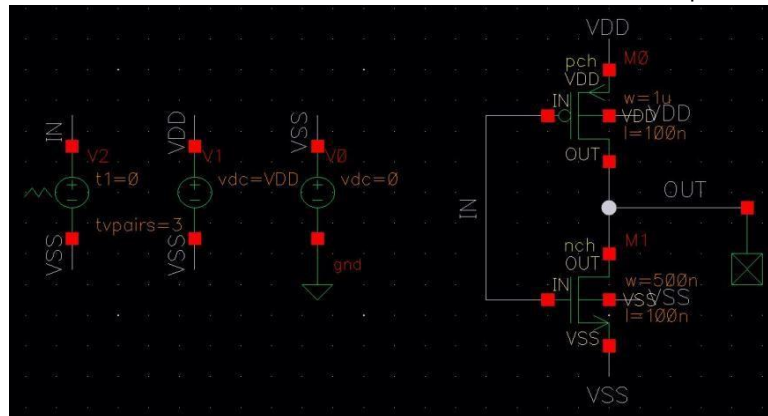


Figure 10: Schematic view of Inverter Cell

Schematic Diagram (Frequently used)	
Commands	Description
w	add a wire
i	add an instance
p	add a pin
l	label to a wire
e	display options like, grid size, snap size etc
q	select an object and press q to open the property dialogue box
{ or shift-z	zoom-out by 2×
} or ctrl-z	zoom-in by 2×
c	copy
m	move: if you move an object, none of the wires connected to it move to maintain connectivity
s	stretch: if you stretch an object all connections to it also extend to maintain connectivity
f3	(when in move or stretch mode): opens dialogue box to select move/stretch mode a) snap mode: any angle. orthogonal, diagonal, etc. When we move an object, it can be restricted to move only orthogonally, or in addition to orthogonal diagonally too, or unrestricted (any angle) this changes those settings b) turn/flip options: If before moving/stretching to final destination you want to rotate or flip the object, then this window (invoked by f3) lets you do that.
f2	save
f8	check and save
u	undo
Shift-U	redo
<delete>	delete an object
ctrl-d	deselect all.
Schematic Diagram (Not frequently used)	
Commands	Description
f3	save
f5	open



tab	pan
f	fit: fits the entire schematic in the window
shift-X	descend to edit by one
b	to go one level up and shift-b to return to top
x	descend to read by one level
ctrl-r	redraw the window
shift-v	world view: see the whole schematic in a small window at bottom right showing which part of it you are at in the main window
ctrl-w	close the window
shift-q	properties of the whole cell view like name etc
Schematic Diagram (Not frequently used)	
Commands	Description
f	fit
shift-f	in hierarchical layout show all levels as if flat
ctrl-f	hide all hierarchy and show only outline of instances
r	rectangle
q	property of an object
ctrl-z	zoom in
shift-z	zoom out
f2	save
t	tap: if you select a layer, saw NW in layout and press tap, that layer gets selected in LSW (layer selection window). Then you can use r to draw rectangles of that layer: Normally we can select a layer in LSW and when we press "r" a rectangle of that layer gets drawn. But if we press "t" (called tap) and then select a shape/rectangle, the layer of that shape/rectangle gets selected in LSW and then pressing r creates rectangle of that layer
p	path: makes a min width path of the layer selected in LSW : If some layer is selected (highlighted) in LSW, then "p" starts to create a path of that layer with width same as the min width for that layer defined in the drc (if it is loaded into icfb)
ctrl-a	select all
ctrl-d	deselect all
c	copy
m	move: move a whole rectangle
s	stretch: can stretch just a side of a rectangle
f	fit
k	ruler
i	add an instance
u	undo
shift-U	redo
shift-r	reshape: use it to reshape a layer so as to make it bigger, e.g., a turn in a metal wire
shift-c	chop: chop a rectangle, i.e., reduce its size as you want
shift-m	merge all rectangles selected as per their layer-purpose name (lpp) (layer purpose pair) for example ("met1" "drawing") is a 2 value array (pair) having value of layer (met1) and its purpose (drawing), i.e. all selected and touching rectangles or paths in same layer-purpose pair (say all met1 drawings) (my above comment will clarify this too) get merged into one.



e	display options like grid size, snap size etc
f6	Redraw
Ctrl- shift-w	make connection using minimum width

Table 2: Cadence Hot-Keys

Simulating a schematic

We want to perform a DC and transient analysis of the system.

9. From the schematic window select **Launch**→**ADE Explorer**. Click OK on the popup window and the **maestro** window should appear. Next select **Variables**→**Copy from Cellview**. The variable "VDD" should appear under the design variables sub window. In the sub window there is nothing under the field "Value" any arbitrary number is needed for the simulation to work, just click the empty box and add "1.2".
10. Next, go to **Analysis**→**Choose**→<select dc>→<check box "save dc operating point">→<click OK>.
11. Similarly for the transient analysis go to **Analysis**→**Choose**→<select tran>→<check box "conservative">→**Stop Time**→<10m>→<click OK>. As shown in figure 11 below.

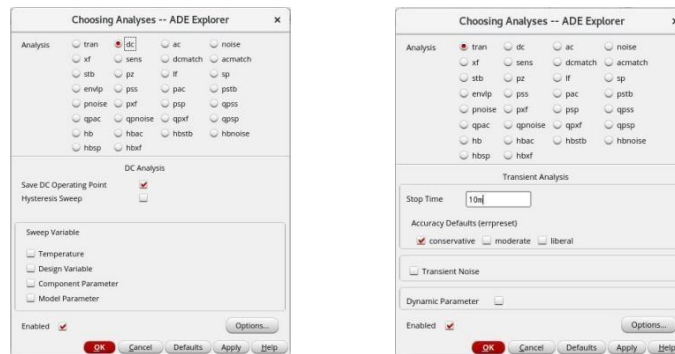


Figure 11: DC and Transient analysis values selection

Next, we need to tell maestro which nets to plot.

12. Go to **Outputs**→**To Be Plotted**→**Select On Design**→<select the target wires>. This process can be repeated with the input and output, just remember to click the wire for voltage because clicking a node will give you its current. Because voltage is desired, wires should be selected not nodes.
13. Next, go back to the maestro window and <check the boxes for "plot" and "save" under the "Outputs" section of the window>. Once this stage is reached the maestro window should look like Figure 12.

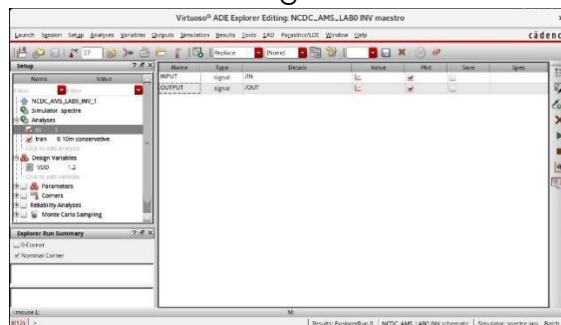


Figure 12: Finalized Maestro window for running simulation



14. Next, **<click the green play button to the right of the window>**. The output plot after clicking the green play button should resemble Figure 13.

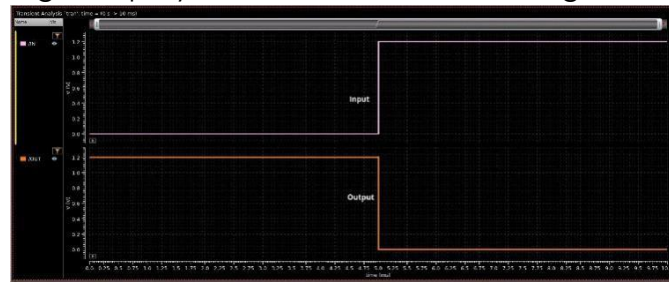


Figure 13: VIVA window

15. Alternatively, you may skip the paragraph above, and just **<click the green play button to the right of the window>** (or click on **Simulation→Netlist and Run** from the maestro menu). After simulation is complete, you may click on the maestro menu item **Results→Direct Plot→Transient Signal**, then select the wires on the schematic window, and press ESC. You will see the same plot window in Figure 13.
16. Similarly, to check the DC node voltages and DC operating points go to **Results→Annotate→DC Node Voltages** and similarly again for operating points go to **Results→Annotate→DC Operating Points**.
17. Now go to schematic window and it will show all the DC node voltages and operating points of all the components as shown in figure 14.

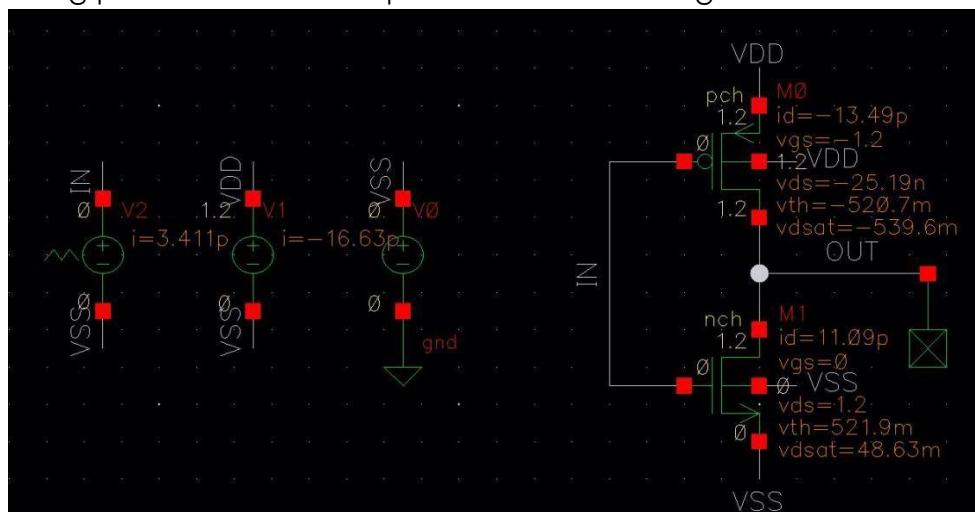


Figure 14: DC_Node_Voltages and DC_Operating_Points

Introduction to TSMC65N PDK

A Process Design Kit (PDK) is a library of basic components generated by the foundry to give access to their process for fabrication. Designers can design a wide variety of integrated circuits (ICs) using the components of the foundry, which are technically and geometrically represented in their Process Design Kits.

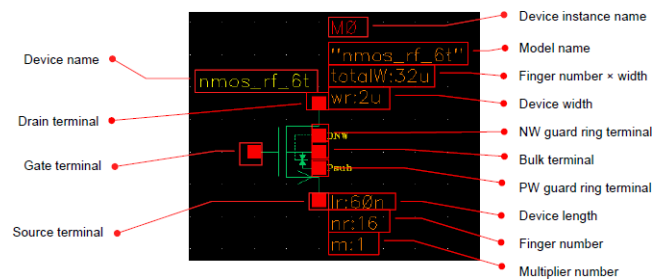
A PDK can be compared to a set of building blocks, where each component in the library is a separate block. A designer can use these blocks to build many types of circuits for various applications. A generic technology is useful for reducing costs when the designer is using predefined, tested components on the material platform of their choice.



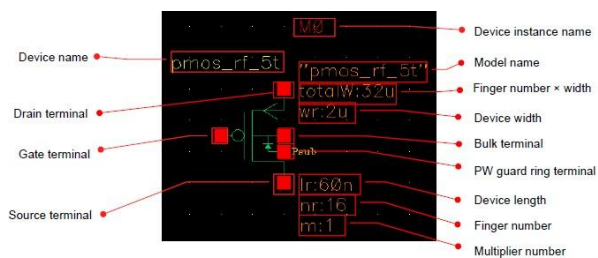
A designer can also create his or her own building blocks, but the designer must follow the fabrication rules of the foundry to be able to use a custom component from a particular foundry. Among others, the rules usually include:

- Material stack (types of layers and thickness)
- Minimum distance between optical components (like gaps between waveguides)
- Maximum etching depths
- Metallization and electrical probes (how to place the metal, metal layers allowed)
- Feature size (size of waveguides, holes, active areas, etc.)

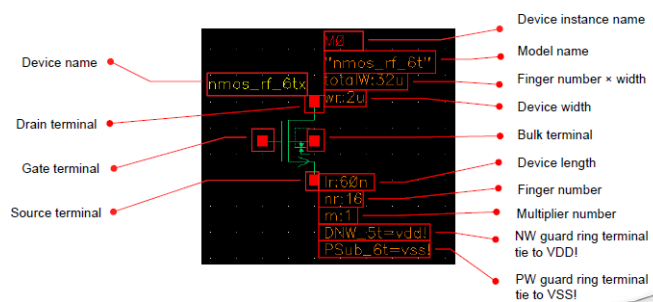
RF_NMOS six terminal



RF_PMOS five terminal

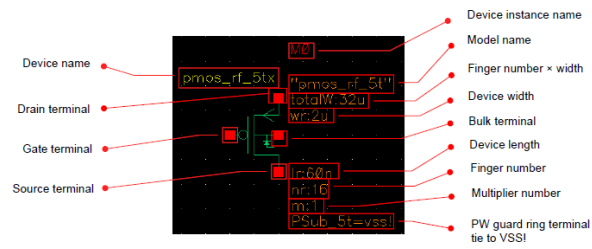


RF_NMOS four terminal

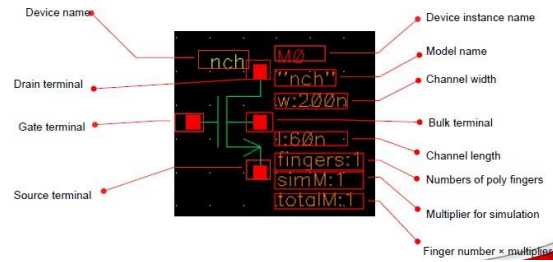




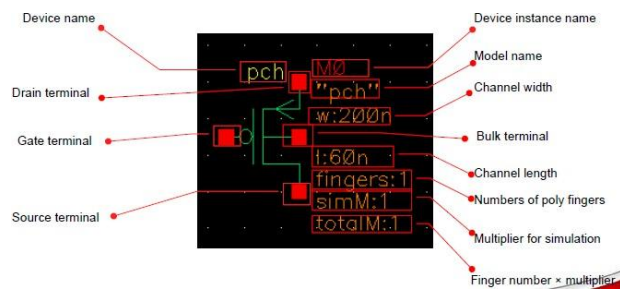
RF_PMOS four terminal



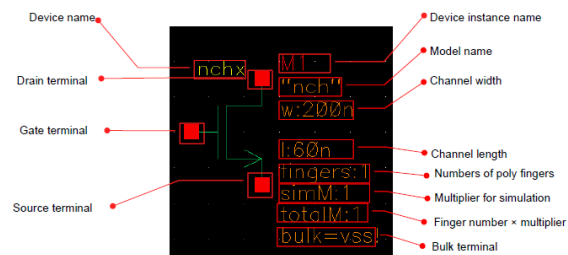
NMOS four terminal



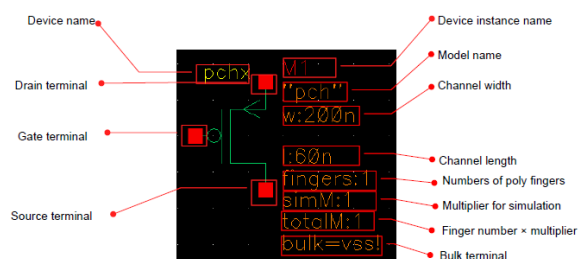
PMOS four terminal



NMOS three terminal

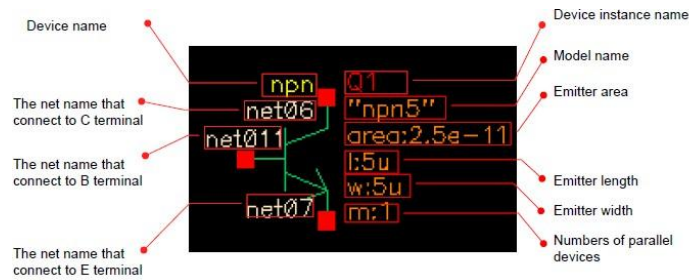


PMOS three terminal

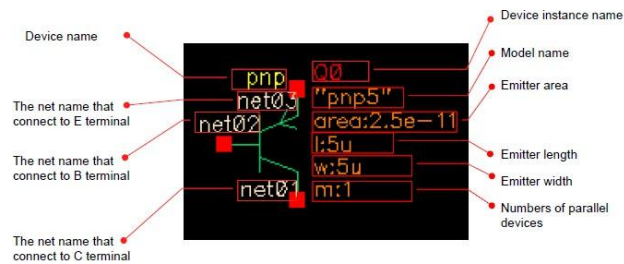




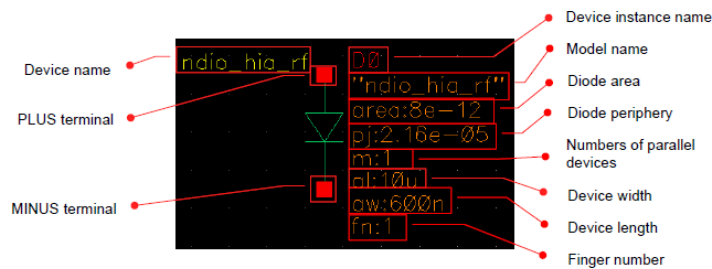
NPN BJT three terminal



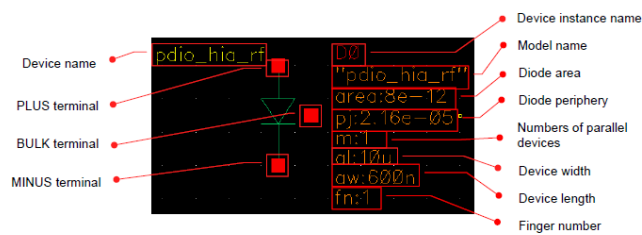
PNP BJT three terminal



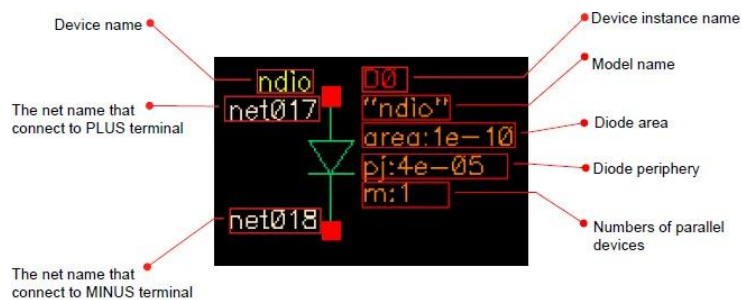
ESD_Diode



ESD_Diode three terminal

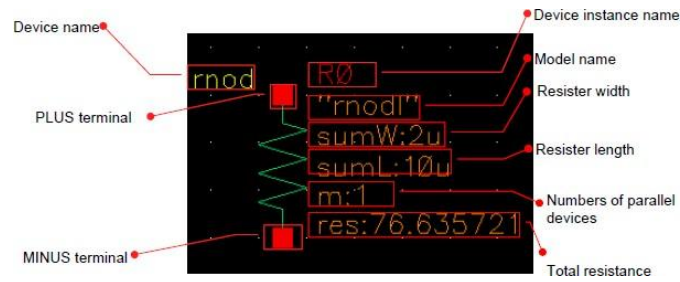


Diode 2 terminal

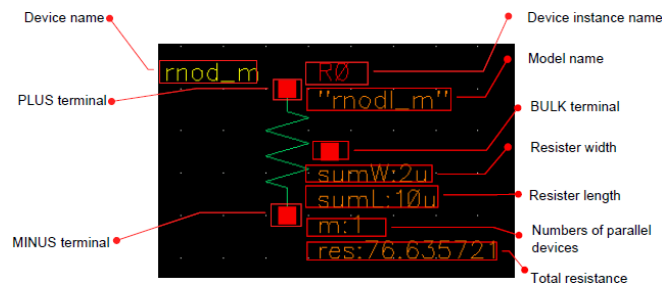




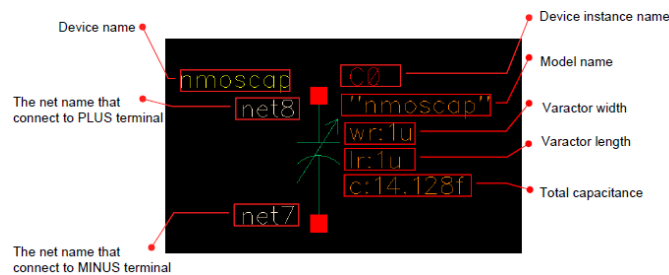
Resistor 2 terminal



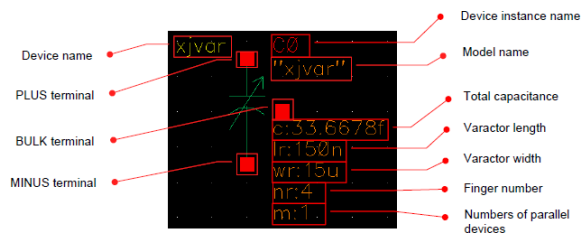
Resistor three terminal



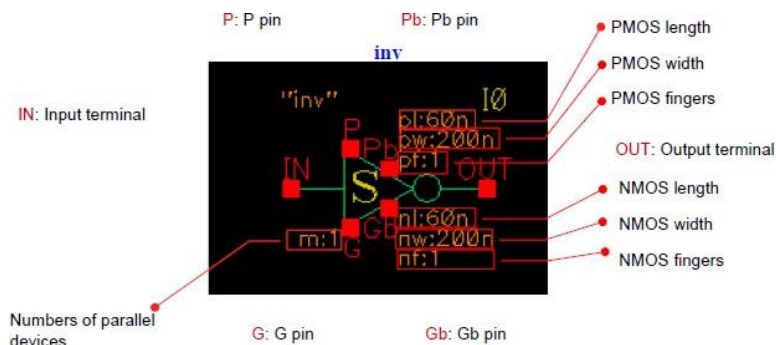
Varactor two terminal



Varactor three terminal

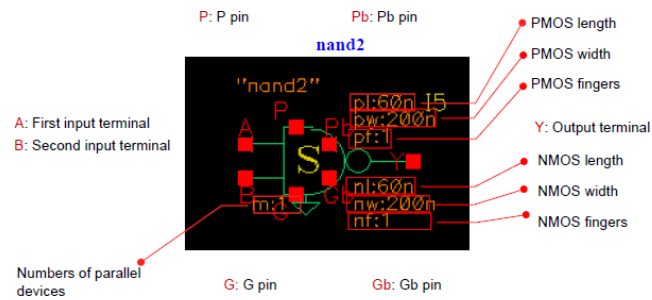


Inverter gate

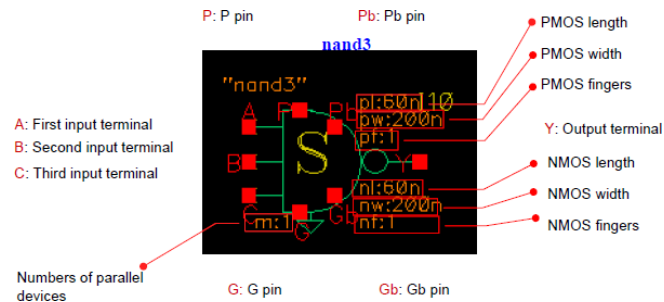




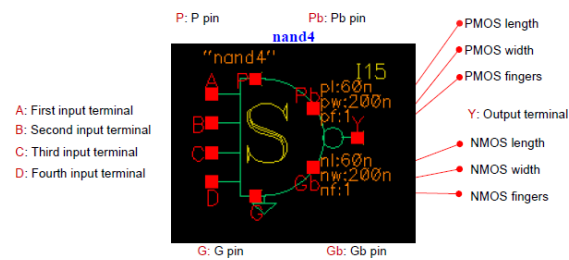
NAND-2T gate



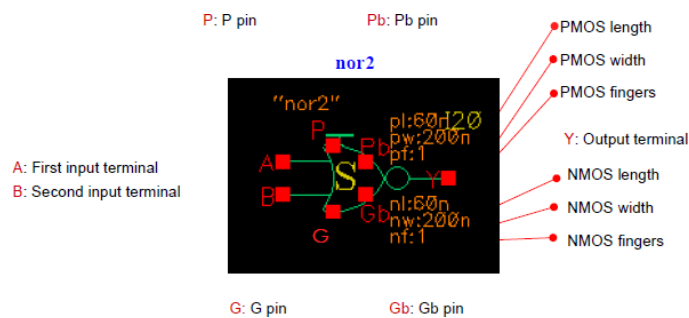
NAND-3T gate



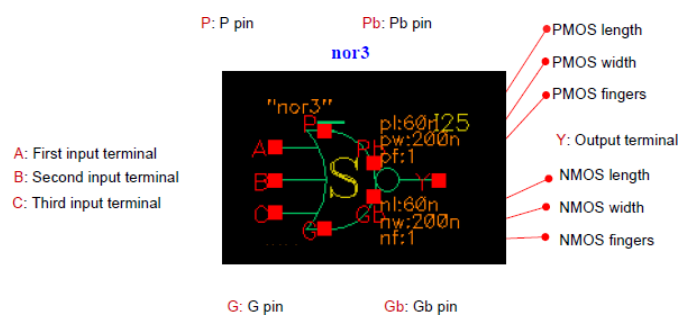
NAND-4T gate



NOR-2T gate

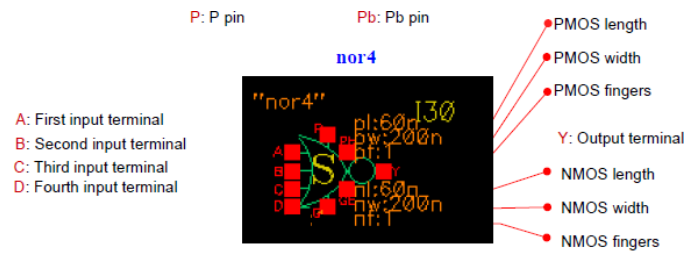


NOR-3T gate

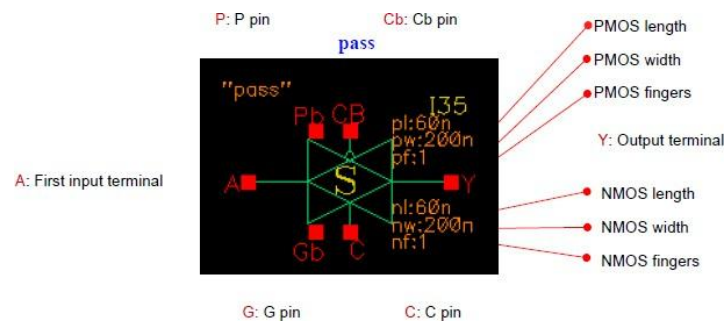




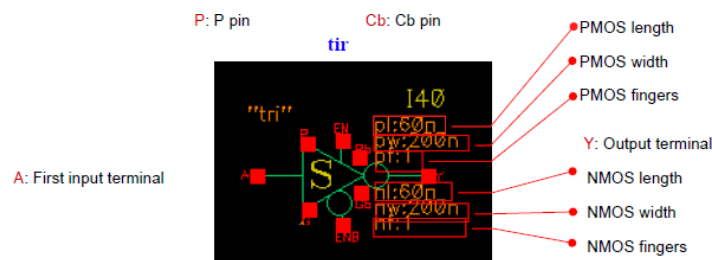
NOR-4T gate



Pass gate



Tri-state inverter gate



Devices available in TSMC65nm PDK

Categories	Device
MOS	nch, nch_dnw, nch_25ud18, nch_25_dnwud18, nch_25, nch_25_dnw, nch_25od33, nch_na25od33, nch_na25od33x,
	nch_25_dnwod33, nch_hvt, nch_hvt_dnw, nch_lvt, nch_lvt_dnw, nch_mlv, nch_mlv_dnw, nch_na, nch_na25, pch,
	pch_25ud18, pch_25, pch_25od33, pch_hvt, pch_lvt, pch_mlv, nch_25ud18x, nch_25x, nch_25od33x, nch_hvtx,
	nch_lvtx, nch_mlvtx, nchx, nch_25_dnwud18x, nch_25_dnw, nch_25_dnwod33x, nch_hvt_dnw, nch_lvt_dnw,
	nch_mlv_dnw, nch_dnw, nch_nax, nch_na25x, pch_25ud18x, pch_25x, pch_25od33x, pch_hvtx, pch_lvtx, pch_mlvtx,
	pchx, pch_hv25, nch_hv25
MOS_MAC	pch_25ud18_mac, pch_25ud18_macx, pch_25_mac, pch_25_macx, pch_25od33_mac, pch_25od33_macx, pch_hvt_mac,
	pch_hvt_macx, pch_lvt_mac, pch_lvt_macx, pch_mac, pch_macx, pch_mlv_mac, pch_mlv_macx, nch_25ud18_mac,



	nch_25ud18_macx, nch_25_mac, nch_25_macx, nch_25od33_mac, nch_25od33_macx, nch_hvt_mac, nch_hvt_macx, nch_lvt_mac, nch_lvt_macx, nch_mac, nch_macx, nch_mlv_t_mac, nch_mlv_t_macx, nch_25_dnwud18_mac, nch_25_dnwud18_macx, nch_25_dnw_mac, nch_25_dnw_macx, nch_25_dnwod33_mac, nch_25_dnwod33_macx, nch_dnw_mac, nch_dnw_macx, nch_hvt_dnw_mac, nch_hvt_dnw_macx, nch_lvt_dnw_mac, nch_lvt_dnw_macx, nch_mlv_t_dnw_mac, nch_mlv_t_dnw_macx, nch_na_mac, nch_na25_mac, nch_na_macx, nch_na25_macx, nch_hv25_mac, pch_hv25_mac
MOS_RF	nmos_rf, nmos_rf_25ud18, nmos_rf_25, nmos_rf_25od33, nmos_rf_hvt, nmos_rf_lvt, nmos_rf_mlv_t, nmos_rf_nodnw, nmos_rf_25_nodnwud18, nmos_rf_25_nodnw, nmos_rf_25_nodnwod33, nmos_rf_hvt_nodnw, nmos_rf_lvt_nodnw, nmos_rf_mlv_t_nodnw, pmos_rf, pmos_rf_25ud18, pmos_rf_25, pmos_rf_25od33, pmos_rf_hvt, pmos_rf_lvt, pmos_rf_mlv_t, pmos_rf_nw, pmos_rf_25_nwud18, pmos_rf_25_nw, pmos_rf_25_nwod33, pmos_rf_hvt_nw, pmos_rf_lvt_nw, pmos_rf_mlv_t_nw, nmos_rf_25ud18_6t, nmos_rf_mlv_t_6t, nmos_rf_6t, nmos_rf_hvt_6t, pmos_rf_25_5t, pmos_rf_25ud18_5t, pmos_rf_25ud18_nw_5t, pmos_rf_lvt_5t, nmos_rf_25od33_6t, pmos_rf_mlv_t_nw_5t, pmos_rf_25od33_5t, pmos_rf_nw_5t, pmos_rf_hvt_5t, pmos_rf_mlv_t_5t, nmos_rf_25_6t, pmos_rf_5t, pmos_rf_hvt_nw_5t, pmos_rf_25_nw_5t, pmos_rf_25od33_nw_5t, nmos_rf_lvt_6t, pmos_rf_lvt_nw_5t, nmos_rf_33_6t, pmos_rf_33_5t, pmos_rf_33_nw_5t
BJT	pnp, pnp_mis, npn, npn_mis, pnp_s, pnp_s_mis, npn_s, npn_s_mis
Diode	ndio, ndio_18, ndio_25, ndio_33, ndio_hvt, ndio_lvt, ndio_mlv_t, ndio_na, ndio_na25, nwdio, pwnw, dnwpsub, dnwpsub, ndio_esd, pdio, pdio_18, pdio_25, pdio_33, pdio_hvt, pdio_lvt, pdio_mlv_t, ndio_hia_rf, pdio_hia_rf, sbd_rf, sbd_rf_nw
Resistor (1)	rm1, rm2, rm3, rm4, rm5, rm6, rm7, rm8, rm9
Resistor (2)	rnod, rnod_m, rnodwo, rnodwo_m, rnpoly, rnpoly_m, rnpolywo, rnpolywo_m, rnwod, rnwod_m, rnwsti, rnwsti_m, rpod, rpod_m, rpodwo, rpodwo_m, rppoly, rppoly_m, rppoly_rf, rppolywo, rppolywo_m, rppolywo_rf, rnod_mx, rnodwo_mx, rnpoly_mx, rnpolywo_mx, rnwod_mx, rnwsti_mx, rpod_mx, rpodwo_mx, rppoly_mx, rppolywo_mx
Capacitors	crtmom, crtmom_mx, crtmom_rf, mimcap, mimcap_3t, mimcap_um_rf, mimcap_woum_rf, mimcap_udc, mimcap_udc_3t, mimcap_um_udc_rf, mimcap_woum_udc_rf
Varactors	nmoscap, nmoscap_25, moscap_rf, moscap_rf_hvt, moscap_rf25, moscap_rf_nw, moscap_rf_hvt_nw, moscap_rf25_nw, xjvar, xjvar_nw, pmoscap_rf, pmoscap_rf25
Inductors	spiral_std_MU_Z, spiral_sym_MU_Z, spiral_sym_ct_MU_Z, spiral_std_MZA_A, spiral_sym_MZA_A, spiral_sym_ct_MZA_A,



Logic Gates	inv, inv_mac, inv_lvt, inv_lvt_mac, inv_hvt, inv_hvt_mac, inv_18, inv_18_mac, inv_25, inv_25_mac, inv_33, inv_33_mac, nand2,
	nand2_mac, nand2_lvt, nand2_lvt_mac, nand2_hvt, nand2_hvt_mac, nand2_18, nand2_18_mac, nand2_25, nand2_25_mac,
	nand2_33, nand2_33_mac, nand3, nand3_mac, nand3_lvt, nand3_lvt_mac, nand3_hvt, nand3_hvt_mac, nand3_18,
	nand3_18_mac, nand3_25, nand3_25_mac, nand3_33, nand3_33_mac, nand4, nand4_mac, nand4_lvt, nand4_lvt_mac,
	nand4_hvt, nand4_hvt_mac, nand4_18, nand4_18_mac, nand4_25, nand4_25_mac, nand4_33, nand4_33_mac, nor2,
	nor2_mac, nor3, nor3_mac, nor3_lvt, nor3_lvt_mac, nor3_hvt, nor3_hvt_mac, nor3_18, nor3_18_mac, nor3_25,
	nor3_25_mac, nor3_33, nor3_33_mac, nor4, nor4_mac, nor4_lvt, nor4_lvt_mac, nor2_lvt, nor2_lvt_mac, nor2_hvt,
	nor2_hvt_mac, nor2_18, nor2_18_mac, nor2_25, nor2_25_mac, nor2_33, nor2_33_mac, nor4_hvt, nor4_hvt_mac, nor4_18,
	nor4_18_mac, nor4_25, nor4_25_mac, nor4_33, nor4_33_mac, tri, tri_mac, tri_lvt, tri_lvt_mac, tri_hvt, tri_hvt_mac, tri_18,
	tri_18_mac, tri_25, tri_25_mac, tri_33, tri_33_mac, pass, pass_mac, pass_lvt, pass_lvt_mac, pass_hvt, pass_hvt_mac,
	pass_18, pass_18_mac, pass_25, pass_25_mac, pass_33, pass_33_mac

Table 3: Devices available in TSMC65nm PDK

Summary

- Began to be familiar with custom IC Design flow
- To login, start a shell tool and start the Cadence Virtuoso software
- To create a working library and get familiar with technology
- Schematic editing tool was then explored and drew a circuit for inverter
- Explored the Maestro for DC and transient analysis of the circuit and schematic nodes annotations
- Learned how to plot simulation graphs in VIVA
- To learn about the PDK and add the PDK library to the library manager
- To know about the short/bind keys that are used in the complete flow



Exercise Tutorial 1

Activity (1.1)

Repeat the exercises above with highest sizes of PMOS and NMOS transistors, observe and generate the current graphs and compare the results with the previous task.

Activity (1.2)

Draw a circuit for 2 input AND gate (CMOS Logic) using tsmc65N nch_mac and pch_mac transistor and observe its output.

Activity (1.3)

Draw a circuit for 2 input OR gate (CMOS Logic) using tsmc65N nch_mac and pch_mac transistor and observe its output.